

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	LOKESH S	Reg. No.:	192572151
Section / Batch:	CSE(AI) /2 ND YEAR	Date:	08/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I LOKESH S (192572151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understand of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. PERFORMANCE ANALYSIS OF MERGE SORT

AIM

To analyze the performance of Merge Sort by executing it for different input sizes and recording the execution time for best, average, and worst cases. The observed results are interpreted using graphs, and the role of the divide-and-conquer approach in maintaining consistent performance is explained.

OBJECTIVE

- Implement Merge Sort using Python.
- Execute the algorithm for different input sizes.
- Measure execution time for best, average, and worst input arrangements.
- Plot the measured execution times.
- Analyze the results using asymptotic complexity.

THEORY

Merge Sort is a comparison-based sorting algorithm that follows the Divide and Conquer strategy. The input array is divided into two approximately equal halves. Each half is recursively sorted and the two sorted halves are then merged into a single sorted array.

The array is divided until each subarray contains one element. Since a one-element array is already sorted, the algorithm then combines the subarrays in sorted order. At every level of recursion, merging requires $O(n)$ work, while the number of levels is $O(\log n)$. Therefore, the overall time complexity is $O(n \log n)$.

COMPLEXITY ANALYSIS

Case	Time Complexity	Explanation
Best Case	$O(n \log n)$	The array is divided and merged regardless of input arrangement.
Average Case	$O(n \log n)$	All elements participate in the merge process.
Worst Case	$O(n \log n)$	The number of divisions and merge levels remains the same.
Space	$O(n)$	Temporary arrays are required during merging.
Stable	Yes	Equal elements can retain their relative order.

ALGORITHM

- If the array contains zero or one element, return it.
- Find the middle position of the array.
- Divide the array into left and right halves.
- Recursively apply Merge Sort to both halves.

- Merge the two sorted halves.
- Return the resulting sorted array.

PYTHON PROGRAM

```
import random
import time
import matplotlib.pyplot as plt

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

sizes = [1000, 5000, 10000, 50000, 100000]

best_times = []
average_times = []
worst_times = []

for n in sizes:
    best = list(range(n))
    average = [random.randint(0, n) for _ in range(n)]
    worst = list(range(n, 0, -1))

    start = time.perf_counter()
```

```

merge_sort(best)
best_times.append(time.perf_counter() - start)

start = time.perf_counter()
merge_sort(average)
average_times.append(time.perf_counter() - start)

start = time.perf_counter()
merge_sort(worst)
worst_times.append(time.perf_counter() - start)

plt.plot(sizes, best_times, marker='o', label='Best')
plt.plot(sizes, average_times, marker='o', label='Average')
plt.plot(sizes, worst_times, marker='o', label='Worst')
plt.xlabel('Input Size')
plt.ylabel('Execution Time (seconds)')
plt.title('Merge Sort Performance')
plt.legend()
plt.grid(True)
plt.show()

```

EXPECTED OBSERVATION

The three cases show broadly similar growth. Unlike algorithms whose behavior depends strongly on the initial ordering, Merge Sort performs the same sequence of divisions and approximately the same amount of merging for every input of a given size. Actual measured times may differ slightly because of system load, memory allocation, Python overhead, and input generation.

INTERPRETATION

The graph should show execution time increasing as input size increases. The growth is approximately $n \log n$ rather than n^2 . The divide-and-conquer method ensures that the problem is repeatedly divided into balanced subproblems, while merging takes linear time at each recursion level.

RESULT

Merge Sort was implemented and analyzed successfully. The experimental behavior is consistent with $O(n \log n)$ time complexity for best, average, and worst cases. Its predictable performance makes it suitable for large datasets where guaranteed performance and stability are important.

2. QUICK SORT WITH DIFFERENT PIVOT SELECTION STRATEGIES

Aim

To evaluate the effectiveness of first, last, and random pivot selection strategies in Quick Sort by comparing their execution times under different input conditions and identifying the strategy that best avoids worst-case behavior.

Theory

Quick Sort is a divide-and-conquer sorting algorithm. It selects a pivot, partitions the input into elements smaller and larger than the pivot, and recursively sorts the partitions. Its performance depends strongly on how balanced the partitions are.

Pivot Strategies

Strategy	Pivot Selected	Risk
First	First element	Poor on sorted/reverse-sorted input
Last	Last element	Poor on sorted/reverse-sorted input
Random	Random element	Greatly reduces predictable worst-case patterns

Complexity

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$
Average auxiliary stack	$O(\log n)$
Worst recursion depth	$O(n)$

Python Program

```
import random
import time
import matplotlib.pyplot as plt
```

```
def quick_sort(arr, strategy):
    if len(arr) <= 1:
        return arr

    if strategy == "first":
        pivot = arr[0]
    elif strategy == "last":
        pivot = arr[-1]
```

```

else:
    pivot = random.choice(arr)

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left, strategy) + middle + quick_sort(right, strategy)

sizes = [1000, 2000, 5000, 10000]
results = {"first": [], "last": [], "random": []}
for n in sizes:
    data = [random.randint(1, n) for _ in range(n)]
    for strategy in results:
        start = time.perf_counter()
        quick_sort(data, strategy)
        elapsed = time.perf_counter() - start
        results[strategy].append(elapsed)
for strategy in results:
    plt.plot(sizes, results[strategy], marker='o', label=strategy)
plt.xlabel("Input Size")
plt.ylabel("Execution Time (seconds)")
plt.title("Quick Sort Pivot Strategy Comparison")
plt.legend()
plt.grid(True)
plt.show()

```

EXPERIMENTAL CONDITIONS

For a meaningful experiment, run the three strategies on identical copies of each dataset. Use random, already sorted, reverse-sorted, and nearly sorted data separately. Repeat each measurement several times and use the average time to reduce random timing fluctuations.

OBSERVATION

On random data, the three strategies can have similar performance. On sorted or reverse-sorted data, selecting the first or last element can repeatedly produce extremely unbalanced partitions. Random pivot selection makes such a pattern unlikely and therefore usually gives more reliable performance.

RESULT

Random pivot selection is generally the most robust of the three simple strategies because it reduces the likelihood of consistently selecting a poor pivot. It does not mathematically eliminate the $O(n^2)$ worst case, but it makes that behavior much less predictable.

3. BINARY SEARCH VS LINEAR SEARCH

AIM

To implement Linear Search and Binary Search on datasets of different sizes, compare their execution times, and justify their suitability based on input conditions and time complexity.

THEORY

Linear Search checks elements sequentially from the beginning until the target is found or the entire dataset has been examined. It does not require sorted data.

Binary Search works only on sorted data. It compares the target with the middle element and discards half of the remaining search interval after each comparison.

COMPLEXITY

Algorithm	Best	Average	Worst	Requirement
Linear Search	$O(1)$	$O(n)$	$O(n)$	No sorting required
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	Data must be sorted

PYTHON PROGRAM

```
import time
import matplotlib.pyplot as plt
```

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
```

```
    while low <= high:
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
```

```

return -1

sizes = [1000, 10000, 100000, 1000000]
linear_times = []
binary_times = []

for n in sizes:
    data = list(range(n))
    target = n - 1

    start = time.perf_counter()
    linear_search(data, target)
    linear_times.append(time.perf_counter() - start)

    start = time.perf_counter()
    binary_search(data, target)
    binary_times.append(time.perf_counter() - start)

plt.plot(sizes, linear_times, marker='o', label='Linear Search')
plt.plot(sizes, binary_times, marker='o', label='Binary Search')
plt.xlabel('Input Size')
plt.ylabel('Execution Time (seconds)')
plt.title('Linear Search vs Binary Search')
plt.legend()
plt.grid(True)
plt.show()

```

OBSERVATION

When the target is placed near the end of a large array, Linear Search performs many comparisons and its execution time grows approximately linearly. Binary Search continues to require only a small number of comparisons because the search interval is halved at each step.

INTERPRETATION

For unsorted or very small datasets, Linear Search is simple and practical. For large sorted datasets, Binary Search is substantially more efficient. If sorting is required solely for a single search operation, the sorting cost must also be considered.

RESULT

Binary Search is the preferred method for repeated searches on sorted large datasets, while Linear Search remains appropriate when the data is unsorted or the dataset is small.

4. STRASSEN'S MATRIX MULTIPLICATION VS STANDARD MATRIX MULTIPLICATION

AIM

To implement Strassen's Matrix Multiplication and standard matrix multiplication, compare their execution times, analyze their time complexities, and identify the situations in which Strassen's algorithm provides computational advantages.

THEORY

Standard matrix multiplication calculates each output element using a dot product. For two $n \times n$ matrices, it performs n^3 scalar multiplications and has $O(n^3)$ time complexity.

Strassen's algorithm uses divide and conquer and reduces the number of recursive matrix multiplications from eight to seven. This changes the recurrence and gives a time complexity of $O(n^{\log_2(7)})$, approximately $O(n^{2.807})$.

COMPLEXITY COMPARISON

Method	Time Complexity	Main Advantage	Main Limitation
Standard	$O(n^3)$	Simple and efficient for small matrices	Higher asymptotic cost
Strassen	$O(n^{2.807})$	Fewer recursive multiplications	More additions, recursion and memory overhead

PYTHON PROGRAM

```
def add(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]

def subtract(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def standard_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def strassen(A, B):
    n = len(A)
    if n == 1:
        return [[A[0][0] * B[0][0]]]
    mid = n // 2
```

```

A11 = [row[:mid] for row in A[:mid]]
A12 = [row[mid:] for row in A[:mid]]
A21 = [row[:mid] for row in A[mid:]]
A22 = [row[mid:] for row in A[mid:]]

B11 = [row[:mid] for row in B[:mid]]
B12 = [row[mid:] for row in B[:mid]]
B21 = [row[:mid] for row in B[mid:]]
B22 = [row[mid:] for row in B[mid:]]

M1 = strassen(add(A11, A22), add(B11, B22))
M2 = strassen(add(A21, A22), B11)
M3 = strassen(A11, subtract(B12, B22))
M4 = strassen(A22, subtract(B21, B11))
M5 = strassen(add(A11, A12), B22)
M6 = strassen(subtract(A21, A11), add(B11, B12))
M7 = strassen(subtract(A12, A22), add(B21, B22))

C11 = add(subtract(add(M1, M4), M5), M7)
C12 = add(M3, M5)
C21 = add(M2, M4)
C22 = add(subtract(add(M1, M3), M2), M6)

C = []
for i in range(mid):
    C.append(C11[i] + C12[i])
for i in range(mid):
    C.append(C21[i] + C22[i])

return C

```

EXPERIMENTAL ANALYSIS

Use square matrices whose dimensions are powers of two, such as 2, 4, 8, 16, 32, 64 and 128, for a simple implementation. Measure both algorithms using the same matrices. For larger matrices, the asymptotic advantage of Strassen can become visible, but a pure Python implementation may still be slower at modest sizes because of list creation, recursion, and matrix addition overhead.

RESULT

Strassen's algorithm has a better theoretical asymptotic complexity than standard multiplication. Its practical advantage generally appears only when matrices are sufficiently large and the implementation is optimized. For small matrices, standard multiplication can be faster because it has much lower overhead.

5. TRADE-OFFS BETWEEN MERGE SORT AND QUICK SORT

AIM

To compare Merge Sort and Quick Sort based on execution time, space complexity, stability, and behavior under different input conditions, and to determine which algorithm is more suitable for large-scale applications.

COMPARISON

Feature	Merge Sort	Quick Sort
Best Time	$O(n \log n)$	$O(n \log n)$
Average Time	$O(n \log n)$	$O(n \log n)$
Worst Time	$O(n \log n)$	$O(n^2)$
Auxiliary Space	$O(n)$	$O(\log n)$ average for in-place recursion
Stable	Yes	Usually No
In-place	Usually No	Usually Yes
Worst-case guarantee	Yes	No, without special safeguards

EXPERIMENTAL PROGRAM

```
import random
import time

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
```

```

return result

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + equal + quick_sort(right)

sizes = [1000, 5000, 10000, 50000]

for n in sizes:
    data = [random.randint(1, n) for _ in range(n)]

    start = time.perf_counter()
    merge_sort(data)
    merge_time = time.perf_counter() - start

    start = time.perf_counter()
    quick_sort(data)
    quick_time = time.perf_counter() - start

    print(n, merge_time, quick_time)

```

INTERPRETATION

Merge Sort offers a guaranteed $O(n \log n)$ upper bound and is stable, but it needs additional memory for merging. Quick Sort can be faster in practice and has lower auxiliary memory when implemented in-place, but its performance depends on pivot selection and partition quality.

RESULT

For large-scale applications requiring stable sorting and predictable worst-case performance, Merge Sort is a strong choice. Quick Sort is attractive when average-case speed and low auxiliary memory are more important and a robust pivot strategy is used.

6. RECURSIVE VS ITERATIVE BINARY SEARCH

AIM

To analyze Recursive and Iterative Binary Search on datasets of different sizes by comparing execution time and memory usage, and to determine the practical effect of recursion.

THEORY

Both versions use the same binary-search principle and therefore have $O(\log n)$ time complexity. The difference is in how repeated search steps are represented. The recursive version creates function-call stack frames, whereas the iterative version uses a loop and reuses the same stack frame.

COMPLEXITY COMPARISON

Method	Time	Extra Space	Practical Characteristic
Recursive	$O(\log n)$	$O(\log n)$	Simple and elegant but uses call stack
Iterative	$O(\log n)$	$O(1)$	Lower memory overhead and generally practical

PYTHON PROGRAM

```
import time
import tracemalloc
import matplotlib.pyplot as plt

def binary_recursive(arr, low, high, target):
    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_recursive(arr, low, mid - 1, target)
    else:
        return binary_recursive(arr, mid + 1, high, target)

def binary_iterative(arr, target):
    low = 0
    high = len(arr) - 1
```

```

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1

return -1

sizes = [1000, 10000, 100000, 1000000]
recursive_times = []
iterative_times = []

for n in sizes:
    data = list(range(n))
    target = n - 1
    start = time.perf_counter()
    binary_recursive(data, 0, n - 1, target)
    recursive_times.append(time.perf_counter() - start)
    start = time.perf_counter()
    binary_iterative(data, target)
    iterative_times.append(time.perf_counter() - start)
plt.plot(sizes, recursive_times, marker='o', label='Recursive')
plt.plot(sizes, iterative_times, marker='o', label='Iterative')
plt.xlabel('Input Size')
plt.ylabel('Execution Time (seconds)')
plt.title('Recursive vs Iterative Binary Search')
plt.legend()
plt.grid(True)
plt.show()

```

MEMORY ANALYSIS

The recursive version has recursion depth $O(\log n)$, so its call stack grows logarithmically. The iterative version uses constant auxiliary space, $O(1)$. In practical measurements, the memory difference may be small for ordinary input sizes, and timing can be affected by Python interpreter overhead.

RESULT

Both approaches have $O(\log n)$ time complexity. The iterative approach is generally more suitable for practical applications because it avoids recursive call overhead and requires $O(1)$ auxiliary space.

Recursion is useful when code clarity and a direct recursive formulation are more important.

7. QUICK SORT UNDER WORST-CASE CONDITIONS

AIM

To analyze Quick Sort under worst-case conditions using already sorted and reverse-sorted datasets, compare the execution time with random inputs, and explain how pivot selection causes performance degradation.

THEORY

Quick Sort performs well when the pivot divides the input into reasonably balanced partitions. If the pivot is consistently the smallest or largest element, one partition contains $n-1$ elements and the other contains zero elements. This produces the recurrence $T(n) = T(n-1) + O(n)$, resulting in $O(n^2)$ time.

PYTHON PROGRAM

```
import random
import time
import sys

sys.setrecursionlimit(1000000)

def quick_sort_first(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort_first(left) + [pivot] + quick_sort_first(right)

def quick_sort_random(arr):
    if len(arr) <= 1:
        return arr

    pivot = random.choice(arr)
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort_random(left) + middle + quick_sort_random(right)
```

```

sizes = [1000, 2000, 5000]

datasets = {
    "Sorted": list(range(1, max(sizes) + 1)),
    "Reverse Sorted": list(range(max(sizes), 0, -1)),
    "Random": [random.randint(1, max(sizes)) for _ in range(max(sizes))]
}

for name, base in datasets.items():
    print("\n", name)

    for n in sizes:
        data = base[:n]

        start = time.perf_counter()
        quick_sort_first(data)
        first_time = time.perf_counter() - start

        start = time.perf_counter()
        quick_sort_random(data)
        random_time = time.perf_counter() - start

        print(n, first_time, random_time)

```

EXPECTED OBSERVATION

For sorted data, selecting the first element produces the smallest possible pivot. For reverse-sorted data, it similarly produces highly unbalanced partitions. As input size increases, execution time can grow rapidly. Random pivot selection usually avoids this predictable degradation.

WORST-CASE AVOIDANCE TECHNIQUES

- Randomized pivot selection.
- Median-of-three pivot selection.
- Three-way partitioning for duplicate-heavy inputs.
- Hybrid Quick Sort with insertion sort for small partitions.
- Recursion-depth safeguards in production implementations.

RESULT

Quick Sort can degrade to $O(n^2)$ when deterministic pivot selection repeatedly creates unbalanced partitions. Randomized or median-based pivot selection is therefore preferred for robust practical implementations.

8. SCALABILITY ANALYSIS OF MERGE SORT

AIM

To analyze the scalability of Merge Sort by applying it to datasets of increasing sizes such as 10^3 , 10^4 and 10^5 elements, recording execution time, plotting the growth trend, and interpreting the results using asymptotic analysis.

THEORY

Merge Sort divides the input into two halves recursively and merges the sorted halves. Its recurrence is $T(n) = 2T(n/2) + O(n)$, which solves to $O(n \log n)$. The complexity is independent of whether the original data is sorted, reverse sorted, or random.

PYTHON PROGRAM

```
import random
import time
import matplotlib.pyplot as plt

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

sizes = [10**3, 10**4, 10**5]
times = []
```

```

for n in sizes:
    data = [random.randint(1, n) for _ in range(n)]

    start = time.perf_counter()
    merge_sort(data)
    elapsed = time.perf_counter() - start

    times.append(elapsed)
    print("Input:", n, "Time:", elapsed)

plt.plot(sizes, times, marker='o')
plt.xlabel("Input Size")
plt.ylabel("Execution Time (seconds)")
plt.title("Merge Sort Scalability")
plt.grid(True)
plt.show()

```

OBSERVATION

The execution time increases as the dataset grows. The increase is consistent with $O(n \log n)$. The graph should not show the sudden quadratic degradation associated with a poorly implemented Quick Sort.

INTERPRETATION

The divide-and-conquer approach keeps the subproblem sizes balanced. Every level of recursion processes approximately n elements during merging, and there are $\log n$ levels. Hence the total work grows as $n \log n$.

RESULT

Merge Sort demonstrates predictable scalability and is well suited for large-scale data processing where consistent asymptotic behavior is required.

9. DIVIDE-AND-CONQUER MATRIX MULTIPLICATION

AIM

To implement divide-and-conquer matrix multiplication for matrices of different sizes, compare its execution time with standard iterative multiplication, and analyze the effect of recursive division.

THEORY

In divide-and-conquer matrix multiplication, each matrix is divided into four submatrices. Eight recursive multiplications are then performed and their results are combined. Because eight subproblems of size $n/2$ are generated and the combination takes $O(n^2)$, the recurrence is $T(n) = 8T(n/2) + O(n^2)$, giving $O(n^3)$.

PYTHON PROGRAM

```
def add(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
```

```
def standard_matrix(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
```

```
    return C
```

```
def divide_matrix(A, B):
    n = len(A)

    if n == 1:
        return [[A[0][0] * B[0][0]]]
```

```
    mid = n // 2
```

```
    A11 = [row[:mid] for row in A[:mid]]
    A12 = [row[mid:] for row in A[:mid]]
    A21 = [row[:mid] for row in A[mid:]]
    A22 = [row[mid:] for row in A[mid:]]
```

```
    B11 = [row[:mid] for row in B[:mid]]
    B12 = [row[mid:] for row in B[:mid]]
```

```

B21 = [row[:mid] for row in B[mid:]]
B22 = [row[mid:] for row in B[mid:]]

C11 = add(divide_matrix(A11, B11),
           divide_matrix(A12, B21))
C12 = add(divide_matrix(A11, B12),
           divide_matrix(A12, B22))
C21 = add(divide_matrix(A21, B11),
           divide_matrix(A22, B21))
C22 = add(divide_matrix(A21, B12),
           divide_matrix(A22, B22))

C = []
for i in range(mid):
    C.append(C11[i] + C12[i])
for i in range(mid):
    C.append(C21[i] + C22[i])

return C

```

COMPLEXITY COMPARISON

Method	Time Complexity	Comment
Standard Iterative	$O(n^3)$	Simple triple-loop implementation
Basic Divide and Conquer	$O(n^3)$	Eight recursive multiplications
Strassen	$O(n^{2.807})$	Uses seven recursive multiplications

INTERPRETATION

Basic divide and conquer does not improve the asymptotic complexity because it still performs eight submultiplications. In a high-level language such as Python, repeated slicing, function calls, and temporary matrix creation may make the recursive version slower than the standard iterative version.

RESULT

Divide and conquer provides a useful structural framework for matrix algorithms, but simple recursive division alone does not improve the $O(n^3)$ complexity. Computational advantages require improved multiplication schemes such as Strassen's algorithm or optimized numerical libraries.

10. IMPACT OF DATA DISTRIBUTION ON SORTING PERFORMANCE

AIM

To analyze the effect of data distribution on Merge Sort and Quick Sort by executing both algorithms on random, nearly sorted, and duplicate-heavy datasets, recording execution time, comparing the results, and determining which algorithm is more suitable under different conditions.

THEORY

The initial distribution of data can significantly affect Quick Sort because partition quality depends on the selected pivot. Merge Sort is much less sensitive because it divides the array independently of its ordering and performs systematic merging.

INPUT TYPES

Dataset	Description
Random	Elements are distributed without an obvious order.
Nearly Sorted	Most elements are ordered with a small number of misplaced elements.
Duplicate Heavy	Many elements have the same or a small number of values.

PYTHON PROGRAM

```
import random
import time
import matplotlib.pyplot as plt

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result += left[i:]
    result += right[j:]

    return result
```

```

        j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + equal + quick_sort(right)

n = 10000

random_data = [random.randint(1, n) for _ in range(n)]

nearly_sorted = list(range(n))
for _ in range(n // 100):
    i = random.randint(0, n - 1)
    j = random.randint(0, n - 1)
    nearly_sorted[i], nearly_sorted[j] = nearly_sorted[j], nearly_sorted[i]

duplicate_data = [random.randint(1, 10) for _ in range(n)]

datasets = {
    "Random": random_data,
    "Nearly Sorted": nearly_sorted,
    "Duplicate Heavy": duplicate_data
}

merge_times = []
quick_times = []
for name, data in datasets.items():
    start = time.perf_counter()
    merge_sort(data)
    merge_time = time.perf_counter() - start

    start = time.perf_counter()
    quick_sort(data)

```

```

quick_time = time.perf_counter() - start
merge_times.append(merge_time)
quick_times.append(quick_time)
print(name)
print("Merge Sort:", merge_time)
print("Quick Sort:", quick_time)

```

```

x = range(len(datasets))
plt.bar([i - 0.2 for i in x], merge_times, width=0.4, label='Merge Sort')
plt.bar([i + 0.2 for i in x], quick_times, width=0.4, label='Quick Sort')
plt.xticks(list(x), list(datasets.keys()))
plt.ylabel("Execution Time (seconds)")
plt.title("Sorting Performance for Different Data Distributions")
plt.legend()
plt.show()

```

OBSERVATION

Merge Sort normally shows comparatively consistent behavior across the three input distributions. Quick Sort can perform very well on random data when the pivot is effective, but its performance can be affected by nearly sorted or duplicate-heavy data depending on the pivot and partitioning method.

DUPLICATE-HEAVY DATA

A standard two-way Quick Sort can repeatedly process equal values inefficiently. Three-way partitioning, which separates elements into less-than, equal-to, and greater-than regions, is a better strategy when many duplicates are present.

COMPARISON

Condition	Merge Sort	Quick Sort
Random	Predictable $O(n \log n)$	Usually $O(n \log n)$ average
Nearly Sorted	Predictable $O(n \log n)$	Depends strongly on pivot
Duplicate Heavy	Predictable $O(n \log n)$	Can degrade without 3-way partitioning
Stable Sorting Required	Suitable	Usually unsuitable
Low Auxiliary Memory	Less suitable	Suitable with in-place implementation

RESULT

Merge Sort is more predictable across different data distributions and is a strong choice when stability and guaranteed $O(n \log n)$ performance are required. Quick Sort can be faster in practical applications when a good pivot and partition strategy are used, but it is more sensitive to the structure of the input.